

Data Science Tools And Software

Dr. Laila M. Fatehy

Department Of Mathematics And Computer Science
Alexandria University

Python Machine Learning

SciKit-Learn

Machine Learning in SciKit-Learn

Scikit-learn, also known as sklearn, is an open-source, robust Python machine learning library. It was created to help simplify the process of implementing machine learning and statistical models in Python.

The common theme in SciKit-Learn is the use of consistent interfaces for all machine learning algorithms.

The main interfaces are:

- **Estimator:** Any object that learns from data; it must implement the `fit` method.
- **Predictor:** Any object that can make predictions; it must implement the `predict` method.
- **Transformer:** Any object that can transform data; it must implement the `fit_transform` and `transform` methods.
- **Model:** Any object that can both fit and predict; it must implement both `fit` and `predict` methods.

Supervised Learning with scikit-learn: Case Studies

"The Iris Flower Dataset"

The Iris Flower Dataset

- The Iris Flower Dataset is a particular dataset used for the first time by Sir Ronald Fisher in 1936. It is often also called the Anderson Iris Dataset.
- In this dataset, data from three different species of iris (Iris setosa, virginica Iris, and Iris versicolor) are collected.
- These data correspond to the length and width of the sepals and the length and width of the petals.
- This dataset is currently being used as a good example for many types of analysis.

Classification Example - Decision Tree Classifier

```
import seaborn as sns
from sklearn import tree
from sklearn import metrics
from sklearn.model_selection import train_test_split # train_test_split is a function in sklearn.model_selection

iris = sns.load_dataset('iris')

# Features and target variable
X = iris.drop('species', axis=1)
y = iris['species']

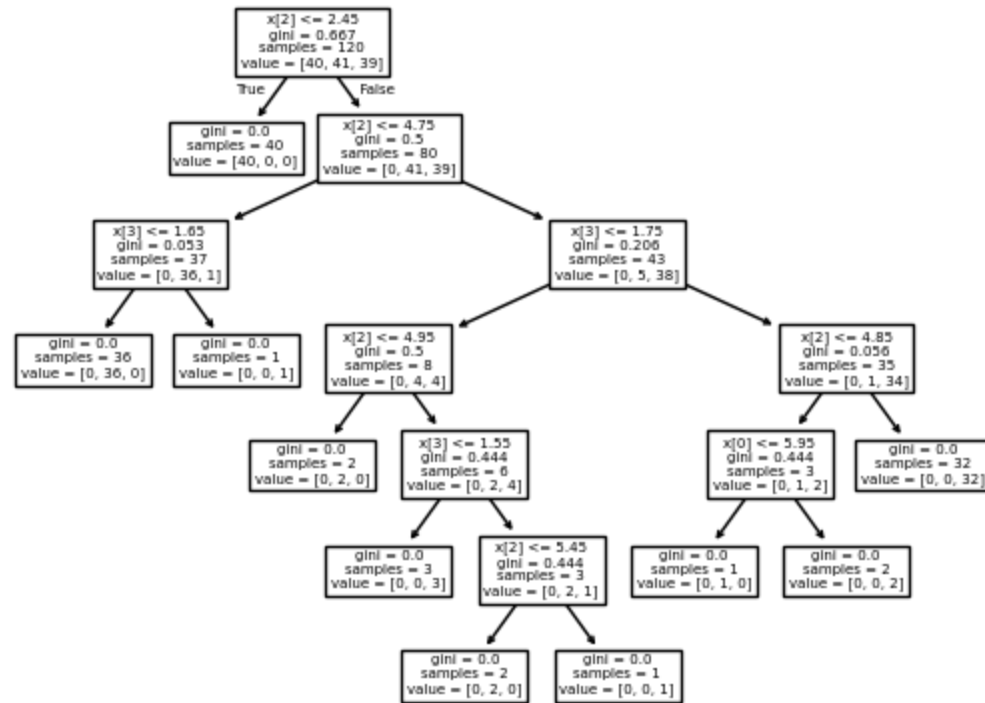
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"X_train: {X_train}")
print(f"X_test: {X_test}")

clf = tree.DecisionTreeClassifier()
clf=clf.fit(X_train, y_train)
tree.plot_tree(clf)

#Predict the response for test dataset
y_pred = clf.predict(X_test)

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
```

Classification Example - Decision Tree Classifier



Accuracy: 1.0

Standardize The Features

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Load the Iris dataset from seaborn
iris = sns.load_dataset('iris')
X = iris.drop('species', axis=1)
y = iris['species']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"X_train: {X_train}")
print(f"X_test: {X_test}")

# Standardize the features
scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Classification Example - RandomForestClassifier

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# Features and target variable
X = iris.drop('species', axis=1)
y = iris['species']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)
rf_clf.fit(X_train_scaled, y_train)
```

Classification Example - RandomForestClassifier

```
# Predictions
y_pred_rf = rf_clf.predict(X_test_scaled)

# Evaluation
print("Random Forest Classification Report:")
print(classification_report(y_test, y_pred_rf))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))
```

```
Random Forest Classification Report:
              precision    recall  f1-score   support

   setosa      1.00      1.00      1.00        10
  versicolor  1.00      1.00      1.00         9
   virginica  1.00      1.00      1.00        11

   accuracy      1.00      1.00      1.00        30
  macro avg      1.00      1.00      1.00        30
weighted avg      1.00      1.00      1.00        30
```

```
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Classification Example - XGBost Classifier

```
import xgboost as xgb
from xgboost import XGBClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix

label_encoder = LabelEncoder()
y_train_encoded = label_encoder.fit_transform(y_train)
y_test_encoded = label_encoder.fit_transform(y_test)
# Initialize and train an XGBoost Classifier
xgb_clf = XGBClassifier(eval_metric='mlogloss', random_state=42)
xgb_clf.fit(X_train_scaled, y_train_encoded)

# Predictions
y_pred_xgb = xgb_clf.predict(X_test_scaled)

# Evaluation
print("XGBoost Classification Report:")
print(classification_report(y_test_encoded, y_pred_xgb))
print("Confusion Matrix:\n", confusion_matrix(y_test_encoded, y_pred_xgb))
```

Classification Example - XGBost Classifier

XGBoost Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Confusion Matrix:

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

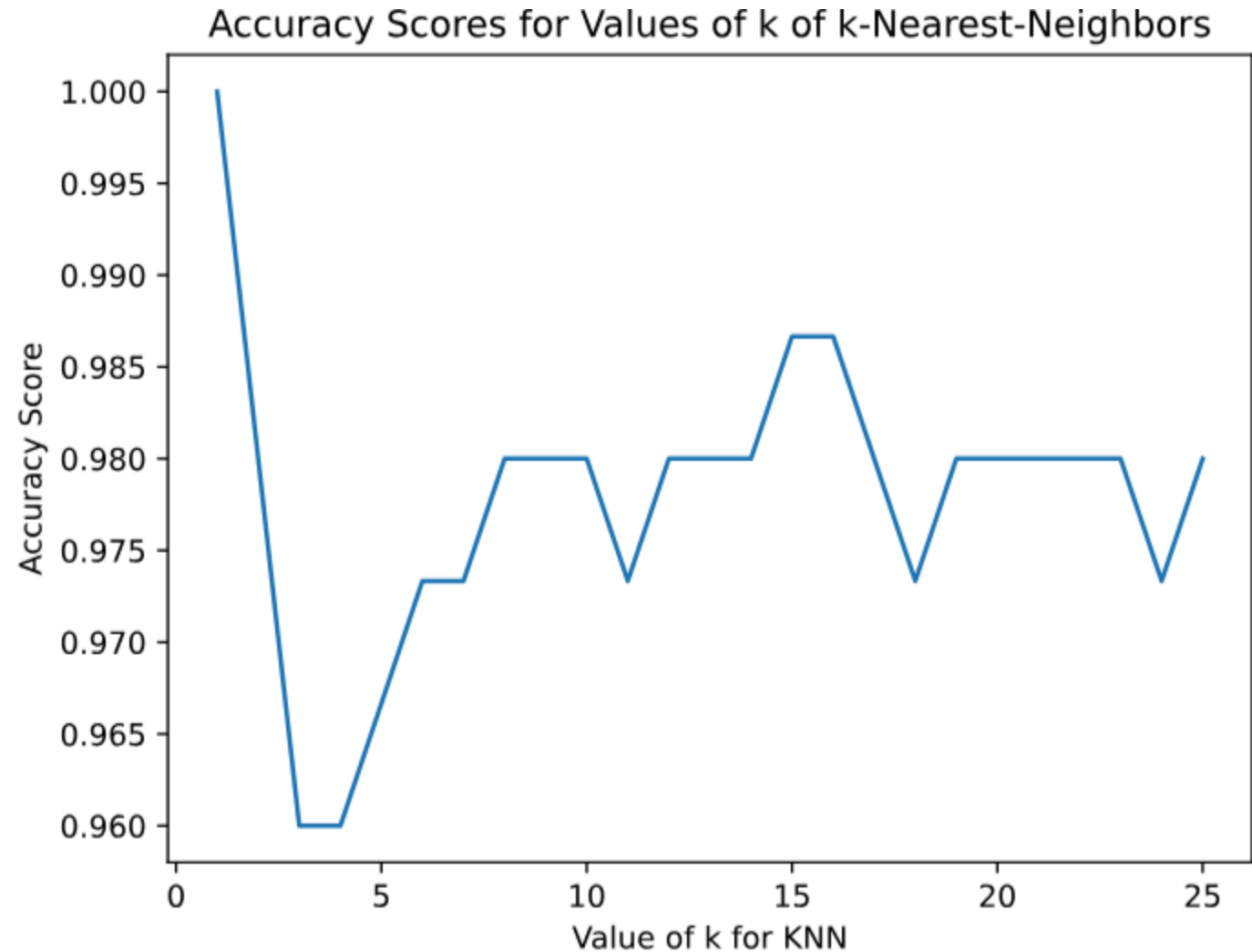
Classification Example - KNN Classifier

```
#KNN Classifier
# experimenting with different n values
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.metrics import classification_report, confusion_matrix

# Selecting features and target variable
X = iris.drop('species', axis=1)
y = iris['species']

k_range = list(range(1,26))
scores = []
for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X, y)
    y_pred = knn.predict(X)
    scores.append(metrics.accuracy_score(y, y_pred))
plt.plot(k_range, scores)
plt.xlabel('Value of k for KNN')
plt.ylabel('Accuracy Score')
plt.title('Accuracy Scores for Values of k of k-Nearest-Neighbors')
plt.show()
```

Classification Example - KNN Classifier



Classification Example - KNN Classifier

```
# Split the dataset into a training set and a testing set

X = iris.drop('species', axis=1)
y = iris['species']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initializing and training the model
model = KNeighborsClassifier(n_neighbors=12)
model.fit(X_train, y_train)

# Making predictions
y_pred = model.predict(X_test)

# Evaluation
print("KNN Classification Report:")
print(classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Classification Example - KNN Classifier

KNN Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Confusion Matrix:

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

- Make a prediction for an example of an out-of-sample observation

```
model.predict([[6, 3, 4, 2]])
```

```
array(['versicolor'], dtype=object)
```

Regression Example - Linear Regression

```
# Import necessary libraries
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Selecting features and target variable
X = iris.drop(['species', 'petal_length'], axis=1)
y = iris['petal_length']

# Splitting the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initializing and training the model
model = LinearRegression()
model.fit(X_train, y_train)
```

Regression Example - Linear Regression

```
# Making predictions
y_pred = model.predict(X_test)

# Evaluating the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse}")
print(f"R-squared: {r2}")
```

```
Mean Squared Error: 1.4018715669865064e-30
R-squared: 1.0
```

Regression Example - Logistic Regression

```
#logistic regression

from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

logreg = LogisticRegression()
logreg.fit(X_train, y_train)
y_pred = logreg.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse}")
print(f"R-squared: {r2}")
```

Types of Logistic Regression

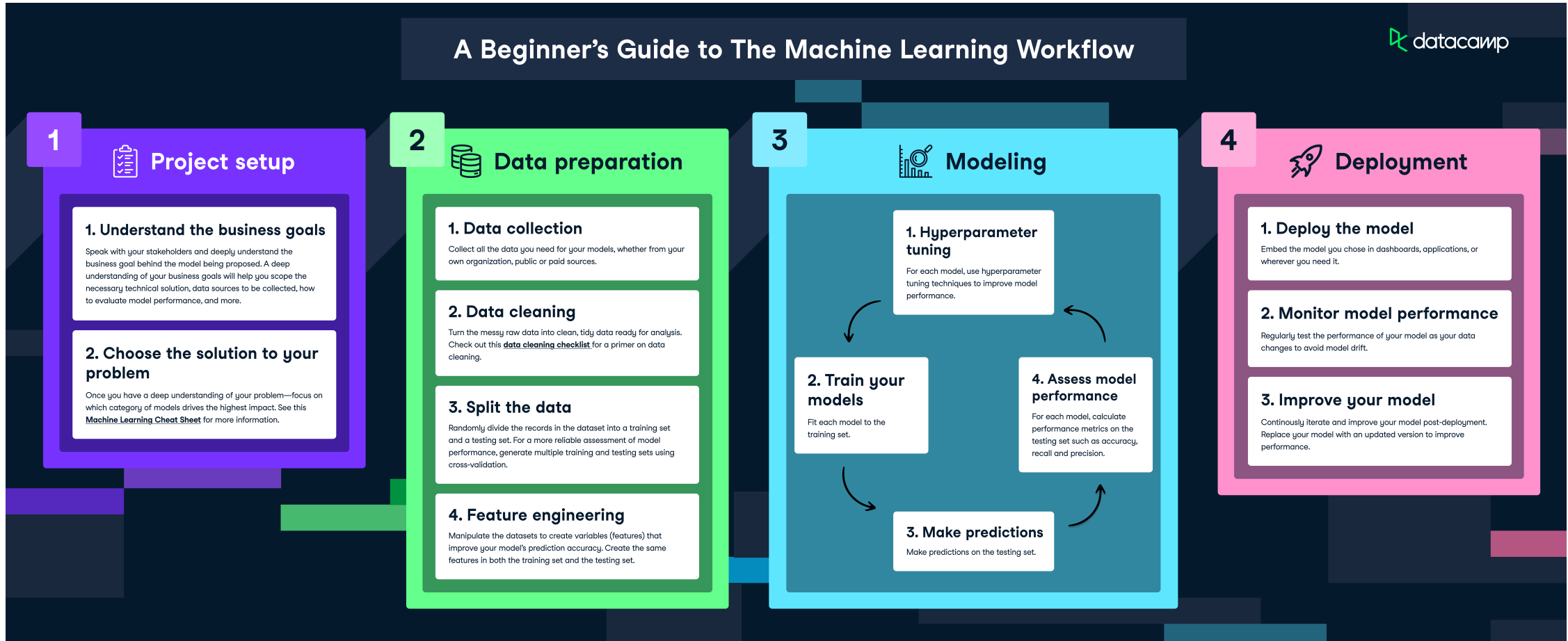
On the basis of the categories, Logistic Regression can be classified into three types:

Binomial: In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.

Multinomial: In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as “cat”, “dogs”, or “sheep”

Ordinal: In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as “low”, “Medium”, or “High”.

A Guide to The Machine Learning Workflow



<https://www.datacamp.com/blog/a-beginner-s-guide-to-the-machine-learning-workflow>

Project Setup - Choose the solution to your problem

Choose the solution to your problem

<https://www.datacamp.com/cheat-sheet/machine-learning-cheat-sheet>

Data preparation - Data cleaning

[Data Cleaning Checklist](#)

<https://www.datacamp.com/blog/infographic-data-cleaning-checklist>